UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Engineering

SUBMITTED BY	**MIAN MUHAMMAD MOEEN**
ROLL NO	**2025(CYS-111-B)**
SECTION	**B**
LAB NO	**5**
SUBMITTED TO	**SIR. HAFIZ MUHAMMAD MUBASHIR**
SEMESTER	**4TH**
SESSION	**2025-2029**
SUBJECT	**ARTIFICIAL INTELLIGENCE**

CYS-241L — LAB 05

TASK 3: INCIDENT-RESPONSE DISPATCH WITH A* SEARCH

Completed solution report | Pakistan CERT road network

Dataset: 27 cities, 43 undirected roads, and coordinates for all 27 cities. Road distances are the approximate values supplied in the lab CSV files.

Part A — Data and heuristic

The roads CSV is loaded as an undirected weighted graph: every listed road is inserted in both directions, and the distance in kilometres is the edge weight. The coordinates CSV is loaded into a city → (latitude, longitude) dictionary.

A1. Straight-line distances to Quetta

City	Haversine distance to Quetta
Lahore	720.30 km
Multan	437.38 km
Karachi	591.46 km
Islamabad	692.81 km
Peshawar	604.47 km

Haversine formula: $a = \sin^2(\Delta\phi/2) + \cos(\phi1)\cos(\phi2)\sin^2(\Delta\lambda/2)$; $d = 2R \arcsin(\sqrt{a})$, where latitude/longitude are converted to radians and R = 6,371 km.

A2. Admissibility check

Program output: None. No road has a straight-line distance greater than its listed road distance.

Explanation: each road length is at least as large as the straight-line distance between its endpoints. By the triangle inequality, the straight-line distance from any city to the goal cannot exceed the length of any road route to that goal. Therefore Haversine is an admissible heuristic for this dataset: it never overestimates the remaining road distance.

Part B — Dijkstra against A*

The shared search function uses a priority queue ordered by f(n) = g(n) + h(n). Setting h(n)=0 makes f(n)=g(n), so the function behaves as Dijkstra's algorithm. With Haversine as h(n), it runs A*. "Expanded cities" counts cities removed from the priority queue and finalized before the goal is reached.

Trip	Dijkstra route	Dijkstra km	Expanded	A* route	A* km	Expanded	Saved
Lahore → Quetta	Lahore → Sahiwal → Multan → Dera Ghazi Khan → Quetta	915	21	Lahore → Sahiwal → Multan → Dera Ghazi Khan → Quetta	915	10	52.4%
Islamabad → Quetta	Islamabad → Faisalabad → Multan → Dera Ghazi Khan → Quetta	1100	23	Islamabad → Faisalabad → Multan → Dera Ghazi Khan → Quetta	1100	16	30.4%
Karachi → Lahore	Karachi → Hyderabad → Sukkur → Rahim Yar Khan → Bahawalpur → Multan → Sahiwal → Lahore	1330	15	Karachi → Hyderabad → Sukkur → Rahim Yar Khan → Bahawalpur → Multan → Sahiwal → Lahore	1330	9	40.0%
Peshawar → Quetta	Peshawar → Kohat → Dera Ismail Khan → Dera Ghazi Khan → Quetta	990	22	Peshawar → Kohat → Dera Ismail Khan → Dera Ghazi Khan → Quetta	990	9	59.1%
Sialkot → Sukkur	Sialkot → Lahore → Sahiwal → Multan → Bahawalpur → Rahim Yar Khan → Sukkur	960	21	Sialkot → Lahore → Sahiwal → Multan → Bahawalpur → Rahim Yar Khan → Sukkur	960	11	47.6%

Average reduction in expanded cities across the five trips: 45.91%. Both algorithms return the same shortest distance for all five trips when A* uses the admissible Haversine heuristic. A* expands fewer cities in every tested trip because the estimate guides the search toward the destination.

Part C — What happens when the heuristic is wrong?

Here the heuristic is doubled: h(n)=2 × Haversine. This can overestimate the remaining road distance, so A* loses its guarantee of optimality.

Trip	Haversine distance km	2×H distance km	Extra km	Shortest preserved?	Route with 2×H
Lahore → Quetta	915	930	15	No	Lahore → Faisalabad → Multan → Dera Ghazi Khan → Quetta
Islamabad → Quetta	1100	1145	45	No	Islamabad → Rawalpindi → Kohat → Dera Ismail Khan → Dera Ghazi Khan → Quetta
Karachi → Lahore	1330	1345	15	No	Karachi → Hyderabad → Sukkur → Rahim Yar Khan → Bahawalpur → Multan → Faisalabad → Lahore
Peshawar → Quetta	990	990	0	Yes	Peshawar → Kohat → Dera Ismail Khan → Dera Ghazi Khan → Quetta
Sialkot → Sukkur	960	960	0	Yes	Sialkot → Lahore → Sahiwal → Multan → Bahawalpur → Rahim Yar Khan → Sukkur

The route is no longer shortest for Lahore → Quetta (+15 km), Islamabad → Quetta (+45 km), and Karachi → Lahore (+15 km). Peshawar → Quetta and Sialkot → Sukkur still happen to return optimal distances in these tests, but this does not make the doubled heuristic admissible or guarantee optimality for other routes.

Answers: (a) An admissible heuristic never overestimates the actual minimum remaining cost. (b) Doubling Haversine makes the search more strongly goal-directed and can expand fewer

cities, but it may prioritize a route that appears promising under the inflated estimate and stop with a more expensive route. (c) When h(n)=0 for every city, A* becomes Dijkstra's algorithm.

Part D — Closed roads and weak points

D1. Close each road on the best Lahore → Quetta route

Original shortest route: Lahore → Sahiwal → Multan → Dera Ghazi Khan → Quetta (915 km). Each road on this route is closed individually, then A* is run again.

Closed road	Road km	New best route	New distance km	Extra km
Lahore — Sahiwal	180	Lahore → Faisalabad → Multan → Dera Ghazi Khan → Quetta	930	15
Sahiwal — Multan	175	Lahore → Faisalabad → Multan → Dera Ghazi Khan → Quetta	930	15
Multan — Dera Ghazi Khan	90	Lahore → Faisalabad → Sargodha → Dera Ismail Khan → Dera Ghazi Khan → Quetta	1185	270
Dera Ghazi Khan — Quetta	470	Lahore → Sahiwal → Multan → Bahawalpur → Rahim Yar Khan → Sukkur → Quetta	1265	350

Most harmful closure: Dera Ghazi Khan — Quetta; it increases the route by 350 km. Almost harmless closures: Lahore — Sahiwal and Sahiwal — Multan; each increases the route by only 15 km because a nearby alternative through Faisalabad is available.

D2. Cities with only one road

City	Only connection	Distance
Skardu	Gilgit	240 km

Skardu is a single point of failure because its only road connects it to Gilgit. If that road closes, Skardu becomes disconnected from the rest of this road network. A network planner should add an alternative connection where geographically feasible, improve contingency transport options, and prepare emergency access plans.

D3. Cybersecurity operation recommendation

Dijkstra is useful when a response team needs shortest distances to many possible incident sites without relying on a goal-directed estimate. A* is useful when the destination is known and a heuristic such as straight-line distance can guide the search. If edge weights represent travel time rather than kilometres, use estimated or measured travel times in minutes for every road. The heuristic should also be in minutes—for example, straight-line distance divided by a defensible maximum speed—so it does not overestimate the remaining travel time.

CODE

```
import csv, io, math, heapq

ROADS = '''origin,destination,distance
Karachi,Hyderabad,165
Karachi,Khuzdar,420
Karachi,Gwadar,640
Hyderabad,Sukkur,330
Sukkur,Quetta,430
Sukkur,Rahim Yar Khan,200
Khuzdar,Quetta,300
Khuzdar,Sukkur,390
Quetta,Dera Ghazi Khan,470
Gwadar,Khuzdar,620
Rahim Yar Khan,Bahawalpur,180
Bahawalpur,Multan,100
Multan,Dera Ghazi Khan,90
Multan,Sahiwal,175
Multan,Faisalabad,240
Dera Ghazi Khan,Dera Ismail Khan,230
Sahiwal,Lahore,180
Faisalabad,Lahore,130
Faisalabad,Sargodha,95
Sargodha,Islamabad,230
Faisalabad,Islamabad,300
Dera Ismail Khan,Sargodha,260
Dera Ismail Khan,Kohat,230
Lahore,Gujranwala,85
Lahore,Sialkot,125
Gujranwala,Sialkot,55
Gujranwala,Jhelum,160
Lahore,Islamabad,375
Jhelum,Rawalpindi,115
```

```
Rawalpindi,Islamabad,15
Islamabad,Peshawar,180
Rawalpindi,Peshawar,175
Peshawar,Mardan,60
Peshawar,Kohat,60
Kohat,Rawalpindi,200
Islamabad,Abbottabad,120
Abbottabad,Muzaffarabad,100
Rawalpindi,Muzaffarabad,140
Mardan,Abbottabad,130
Abbottabad,Gilgit,470
Gilgit,Skardu,240
Mardan,Gilgit,650
Jhelum,Muzaffarabad,200'''

COORDS = '''city,latitude,longitude
Karachi,24.8607,67.0011
Hyderabad,25.396,68.3578
Sukkur,27.7052,68.8574
Quetta,30.1798,66.975
Gwadar,25.1264,62.3225
Multan,30.1575,71.5249
Bahawalpur,29.3956,71.6836
Faisalabad,31.4504,73.135
Lahore,31.5204,74.3587
Sargodha,32.0836,72.6711
Gujranwala,32.1877,74.1945
Sialkot,32.4945,74.5229
Islamabad,33.6844,73.0479
```

```
Peshawar,34.0151,71.5249
Mardan,34.1989,72.0231
Abbottabad,34.1688,73.2215
Dera Ismail Khan,31.8626,70.9019
Muzaffarabad,34.37,73.4711
Gilgit,35.9208,74.308
Skardu,35.2971,75.6339
Sahiwal,30.6682,73.1114
Rahim Yar Khan,28.4202,70.2952
Khuzdar,27.8,66.6167
Dera Ghazi Khan,30.0489,70.6455
Jhelum,32.9425,73.7257
Kohat,33.5869,71.4414'''

graph = {}
for r in csv.DictReader(io.StringIO(ROADS)):
    a, b, w = r['origin'], r['destination'], float(r['distance'])
    graph.setdefault(a, {})[b] = w
    graph.setdefault(b, {})[a] = w
coords = {r['city']: (float(r['latitude']), float(r['longitude']))
          for r in csv.DictReader(io.StringIO(COORDS))}

def h(a, b):
    if a not in coords or b not in coords: return 0
    lat1, lon1 = map(math.radians, coords[a]); lat2, lon2 = map(math.radians, coords[b])
    x = math.sin((lat2-lat1)/2)**2 + math.cos(lat1)*math.cos(lat2)*math.sin((lon2-lon1)/2)**2
    return 12742 * math.asin(min(1, math.sqrt(x)))

def search(g, start, goal, heuristic):
    q = [(heuristic(start, goal), 0, start)]
    dist, parent, expanded = {start: 0}, {}, 0
```

```
    while q:
        f, cost, city = heapq.heappop(q)
        if cost != dist.get(city): continue
        expanded += 1
        if city == goal:
            path = [goal]
            while path[-1] != start: path.append(parent[path[-1]])
            return path[::-1], cost, expanded
        for nxt, weight in g.get(city, {}).items():
            new = cost + weight
            if new < dist.get(nxt, float('inf')):
                dist[nxt], parent[nxt] = new, city
                heapq.heappush(q, (new + heuristic(nxt, goal), new, nxt))
    return None, float('inf'), expanded

print('TASK 3: INCIDENT-RESPONSE DISPATCH')
print('PART A - Dataset:', len(graph), 'cities,', sum(map(len, graph.values()))//2, 'roads')
print('Haversine estimate Lahore to Quetta:', round(h('Lahore','Quetta'), 1), 'km')
violations = [(a,b) for a in graph for b,w in graph[a].items() if a < b and h(a,b) > w + 1e-6]
print('Road-edge heuristic violations:', violations if violations else 'None')

trips = [('Lahore','Quetta'), ('Islamabad','Quetta'), ('Karachi','Lahore'), ('Peshawar','Quetta'), ('Sial
print('\nPART B - Dijkstra vs A*')
for s, t in trips:
    d = search(graph, s, t, lambda a,b: 0)
    a = search(graph, s, t, h)
    print(f'{s} -> {t}\n Dijkstra: {d[1]:.0f} km, {d[2]} expanded; path: {" -> ".join(d[0]) if d[0] else
    print(f' A*:       {a[1]:.0f} km, {a[2]} expanded; path: {" -> ".join(a[0]) if a[0] else "No route"}')
```

```
    print(f' A*:       {a[1]:.0f} km, {a[2]} expanded; path: {" -> ".join(a[0]) if a[0] else "No route"}')

print('\nPART C - Overestimating heuristic (2 x Haversine)')
for s, t in trips:
    normal = search(graph, s, t, h); over = search(graph, s, t, lambda a,b: 2*h(a,b))
    print(f'{s} -> {t}: normal={normal[1]:.0f} km, doubled={over[1]:.0f} km')

print('\nPART D1 - Close roads on the Lahore-Quetta A* route')
path, original, _ = search(graph, 'Lahore', 'Quetta', h)
print('Original:', ' -> '.join(path), f'({original:.0f} km)')
for a, b in zip(path, path[1:]):
    w = graph[a].pop(b); graph[b].pop(a)
    alt = search(graph, 'Lahore', 'Quetta', h)
    print(f'Close {a}-{b}:', ' -> '.join(alt[0]) if alt[0] else 'No alternative route', '| distance:', alt[1])
    graph[a][b] = graph[b][a] = w

print('\nPART D2 - Cities with only one road')
leaves = [c for c in graph if len(graph[c]) == 1]
for c in leaves: print(c, '->', next(iter(graph[c])))
if not leaves: print('None')

print('\nPART D3 - Cybersecurity application')
print('A* can help incident-response teams plan a route to a known destination.')
print('Dijkstra finds shortest-distance routes without a heuristic. Update the graph when roads close.')
```

OUTPUT

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\LENOVO\Downloads> & 'C:\Users\LENOVO\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\LENOVO\.
code\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63410' '--' 'c:\Users\LENOVO\Downlo
s\task3_standalone.py'
TASK 3: INCIDENT-RESPONSE DISPATCH
PART A - Dataset: 27 cities, 43 roads
Haversine estimate Lahore to Quetta: 720.3 km
Road-edge heuristic violations: None

PART B - Dijkstra vs A*
Lahore -> Quetta
 Dijkstra: 915 km, 21 expanded; path: Lahore -> Sahiwal -> Multan -> Dera Ghazi Khan -> Quetta
 A*:       915 km, 10 expanded; path: Lahore -> Sahiwal -> Multan -> Dera Ghazi Khan -> Quetta
Islamabad -> Quetta
```

```
Problems  Output  Debug Console  Terminal  Ports

  A*:       915 km, 10 expanded; path: Lahore -> Sahiwal -> Multan -> Dera Ghazi Khan -> Quetta
 Islamabad -> Quetta
  Dijkstra: 1100 km, 23 expanded; path: Islamabad -> Faisalabad -> Multan -> Dera Ghazi Khan -> Quetta
  A*:       1100 km, 16 expanded; path: Islamabad -> Faisalabad -> Multan -> Dera Ghazi Khan -> Quetta
 Karachi -> Lahore
  Dijkstra: 1330 km, 15 expanded; path: Karachi -> Hyderabad -> Sukkur -> Rahim Yar Khan -> Bahawalpur -> Multan -> Sahiw
 al -> Lahore
  A*:       1330 km, 9 expanded; path: Karachi -> Hyderabad -> Sukkur -> Rahim Yar Khan -> Bahawalpur -> Multan -> Sahiwa
 l -> Lahore
 Peshawar -> Quetta
  Dijkstra: 990 km, 22 expanded; path: Peshawar -> Kohat -> Dera Ismail Khan -> Dera Ghazi Khan -> Quetta
  A*:       990 km, 9 expanded; path: Peshawar -> Kohat -> Dera Ismail Khan -> Dera Ghazi Khan -> Quetta
 Sialkot -> Sukkur
```

```
Close Lahore-Sahiwal: Lahore -> Faisalabad -> Multan -> Dera Ghazi Khan -> Quetta | distance: 930.0
Close Sahiwal-Multan: Lahore -> Faisalabad -> Multan -> Dera Ghazi Khan -> Quetta | distance: 930.0
ce: 1185.0
Close Dera Ghazi Khan-Quetta: Lahore -> Sahiwal -> Multan -> Bahawalpur -> Rahim Yar Khan -> Sukkur -> Quetta | distance

PART D2 - Cities with only one road
Skardu -> Gilgit
PART D3 - Cybersecurity application
A* can help incident-response teams plan a route to a known destination.
PS C:\Users\LENOVO\Downloads> ^C
PS C:\Users\LENOVO\Downloads>
PS C:\Users\LENOVO\Downloads>  c:; cd 'c:\Users\LENOVO\Downloads'; & 'C:\Users\LENOVO\AppData\Local\Programs\Python\Pyth
on313\python.exe' 'c:\Users\LENOVO\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher
```